Ciontu Claudia-Elena
Draghici Andreea-Maria
CR4.S1A

Tema 8 – Bile Saltarete

Laborator Sisteme de Prelucrare Grafică

2023

Ciontu Claudia-Elena
Draghici Andreea-Maria
CR4.S1A

Cuprins:

- **PREZENTAREA INTERMEDIARA .**
- **PREZENTAREA FINALA.**

Ciontu Claudia-Elena
Draghici Andreea-Maria
CR4.S1A

PREZENTAREA INTERMEDIARA

Initial am adaugat librariile de care avem nevoie pentru a crea o bila 3D ce are ca textura conceptul de sfera. Pentru a randa 3D sfera am utilizat anumite metode din libraria GLUT, cat si OpenGL.

```
#include <GLFW/glfw3.h>
#include <stdlib.h>
#include <stdio.h>
#include <math.h>
#include <GL/gl.h>
#include <iostream>
#include <GL/glut.h>
```

Pentru a adauga detalii suplimentare la bila (exemplu luminozitate, contrast etc) am utilizat texturi. Codul de mai jos este o initializare a unui set de variabile globale utilizate pentru a desena bila in scena, de o anima si de aschimba parametrii de iluminare pentru a obtine efecte vizuale diferite.

```
using namespace std;

// variabile utilizate pentru stocarea dimensiunii ferestrei grafice
int windowWidth = 800;
int windowHeight = 600;

// variabile utilizate pentru a reda unghiurile de rotatie ale camerei in jurul axelor
float cameraAngleX = 0.0f;
float cameraAngleY = 0.0f;

// reprezinta distanta dintre camera si obiect
float cameraDistance = 10.0f;

// reprezinta pozitia curenta a bilei in spatiul 3D
float ballX = 0.0f;
float ballY = 0.0f;
float ballZ = 0.0f;

// reprezinta vitezele de deplasare ale bilei in cele trei axe ale spatiului 3D
float ballDX = 0.1f;
float ballDY = 0.05f;
float ballDZ = 0.2f;

// raza bilei
float ballRadius = 0.5f;

// reprezinta parametrii luminii folositi in calcularea aspectului obiecului
float lightPosition[4] = { 0.0f, 5.0f, 0.0f, 1.0f };
float ambientLight[4] = { 0.2f, 0.2f, 0.2f, 1.0f };
float diffuseLight[4] = { 1.0f, 1.0f, 1.0f, 1.0f };
float specularLight[4] = { 1.0f, 1.0f, 1.0f, 1.0f };

// utilizata pentru a controla stralucirea suprafetei bilei
float shininess = 100.0f;

// identificator pentru o textura de decorare a obiectului
GLuint textureId;
```

Ciontu Claudia-Elena
Draghici Andreea-Maria
CR4.S1A

Acest cod utilizeaza iluminarea din OpenGL si texturarea pentru a face sfera sa para mult mai realista. Textura consta intr-un gradient de culoare de la rosu la alb.

```
/* functie care are ca scop incarcarea unei texturi in memoria GPU pentru a desena obiectul */
void loadTexture() {

    // tablou de 16 octeti (4 pixeli, fiecare cu 4 canale RGBA) care reprezinta continutul texturii.
    unsigned char data[] = { 255, 0, 0, 0, 150, 0, 0, 150, 255, 150, 150, 150, 255, 150, 150, 255 };

    // generarea unui identificator de textura
    glGenTextures(n: 1, textures: &textureId);

    // folosita pentru a asocia textura identificata cu un tip de textura specific
    glBindTexture(target: GL_TEXTURE_2D, textureId);

    // utilizata pentru a specifica continutul texturii si pentru a incarca datele in memoria GPU.
    glTexImage2D(target: GL_TEXTURE_2D, level: 0, internalformat: GL_RGBA, width: 2, height: 2, border: 0, format: GL_RGBA, type: GL_UNSIGNED_BYTE, pixels: data);

    // utilizate pentru a seta parametrii de filtrare pentru textura, pentru a controla modul in care textura
    // este redimensionata sau estompata atunci cand este afisata la dimensiuni diferite decat cele originale
    glTexParameteri(target: GL_TEXTURE_2D, pname: GL_TEXTURE_MAG_FILTER, param: GL_LINEAR);
    glTexParameteri(target: GL_TEXTURE_2D, pname: GL_TEXTURE_MIN_FILTER, param: GL_LINEAR);
}
```

```
/* functie pentru a afisare */
void display() {

    // se curata buffer-ul de culoare
    glClear(mask: GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    // seteaza matricea de modelare si vizualizare
    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();

    // folosita pentru a defini pozitia camerei cu ajutorul parametrilor de distanta , unghi de vedere si pozitia obiectului
    gluLookAt(eyex: cameraDistance * sin(_Xx: cameraAngleY) * cos(_Xx: cameraAngleX),
        eyey: cameraDistance * sin(_Xx: cameraAngleX),
        eyez: cameraDistance * cos(_Xx: cameraAngleY) * cos(_Xx: cameraAngleX),
        centerx: 0.0, centery: 0.0, centerz: 0.0,
        upx: 0.0, upy: 1.0, upz: 0.0);
    glPushMatrix();

    // se aplica transformarea de translatie si rotatie pentru a plasa bila in pozitia si orientarea dorita
    glTranslatef(ballX, ballY, ballZ);
    glRotatef(angle: 30.0f, x: 1.0f, y: 1.0f, z: 1.0f);

    // se activeaza textura
    glEnable(cap: GL_TEXTURE_2D);

    // se ataseaza textura bilei
    glBindTexture(target: GL_TEXTURE_2D, textureId);

    // se activeaza iluminarea
    glEnable(cap: GL_LIGHTING);

    // se defineste sursa de lumina
    glEnable(cap: GL_LIGHT0);
    glLightfv(GL_LIGHT0, pname: GL_POSITION, params: lightPosition);
    glLightfv(GL_LIGHT0, pname: GL_AMBIENT, params: ambientLight);
    glLightfv(GL_LIGHT0, pname: GL_DIFFUSE, params: diffuseLight);
    glLightfv(GL_LIGHT0, pname: GL_SPECULAR, params: specularLight);

    // definire material pentru bila
    glMaterialfv(face: GL_FRONT, pname: GL_AMBIENT_AND_DIFFUSE, params: diffuseLight);
    glMaterialfv(face: GL_FRONT, pname: GL_SPECULAR, params: specularLight);
    glMaterialf(face: GL_FRONT, pname: GL_SHININESS, param: shininess);

    // desenare sfera
    glutSolidSphere(ballRadius, slices: 50, stacks: 50);

    // dezactivare textura si iluminare
    glDisable(cap: GL_TEXTURE_2D);
    glDisable(cap: GL_LIGHTING);
    glPopMatrix();

    // schimbare buffer pentru a permite afisarea
    glutSwapBuffers();
}
```

Ciontu Claudia-Elena
Draghici Andreea-Maria
CR4.S1A

```
/* apelata cand fereastra de afisare este redimensionata
   scop pentru a adapta proiectia proportiilor obiectului afisat pentru a nu se deforma in cazul schimbarii dimensiunii ferestrei */
void reshape(int w, int h) {

    // specificam noua zona de afisare a ferestrei, aceasta fiind impartita in regiuni de coordonate
    glViewport(x: 0, y: 0, width: w, height: h);

    // se seteaza matricea de proiectie
    glMatrixMode(mode: GL_PROJECTION);
    glLoadIdentity();

    // se specifica campul vizual al camerei in grade, raportul de aspect al ferestrei,
    // precum si distanta dintre planul apropiat si cel indepartat
    gluPerspective(fovy: 45.0, aspect: (float)w / (float)h, zNear: 0.1, zFar: 100.0);
}
```

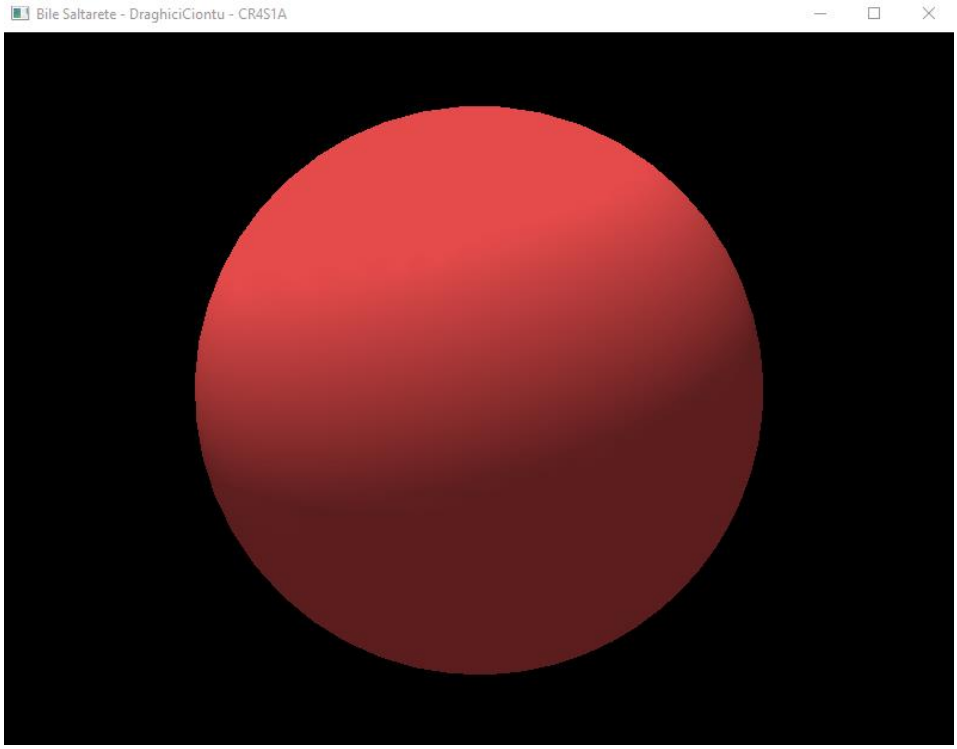
```
/* gestionarea tastelor pentru a controla pozitia bilei , primind ca caracter tasta apasata si coordonatele cursorului la momentul apasarii tastei */
void keyboard(unsigned char key, int x, int y) {

    // verifica ce tasta a fost apasata si executa actiunea corespunzatoare
    switch (key) {
    case 'w':
        ballY += ballDY; // bila se ridica pe axa Y
        break;
    case 's':
        ballY -= ballDY; // bila coboara pe axa Y
        break;
    case 'a':
        ballX -= ballDX; // bila se va deplasa spre stanga pe axa X
        break;
    case 'd':
        ballX += ballDX; // bila se va deplasa spre dreapta pe axa X
        break;
    case 'q':
        ballZ -= ballDZ; // bila se va apropia de observator pe axa Z
        break;
    case 'e':
        ballZ += ballDZ; // bila se va indeparta de observator pe axa Z
        break;
    case 'r':
        // reintoarcere la pozitia initiala
        ballX = 0.0f;
        ballY = 0.0f;
        ballZ = 0.0f;
        break;
    case 27:
        exit(_Code: 0); // apasare tasta ESC iar programul se inchide
        break;
    default:
        break;
    }
}
```

```
/* pentru tastele speciale */
void special_keyboard(int key, int x, int y) {

    // pentru a determina care tasta a fost apasata
    switch (key) {
    case GLUT_KEY_UP:
        cameraAngleX -= 0.1f; // se decrementeaza unghiul de rotatie al camerei in jurul axei X
        break;
    case GLUT_KEY_DOWN:
        cameraAngleX += 0.1f; // se incrementeaza unghiul de rotatie al camerei in jurul axei X
        break;
    case GLUT_KEY_LEFT:
        cameraAngleY -= 0.1f; // se decrementeaza unghiul de rotatie al camerei in jurul axei Y
        break;
    case GLUT_KEY_RIGHT:
        cameraAngleY += 0.1f; // se incrementeaza unghiul de rotatie al camerei in jurul axei Y
        break;
    default:
        break;
    }
}
```

Ciontu Claudia-Elena
Draghici Andreea-Maria
CR4.S1A

```
// main function
int main(int argc, char** argv) {

    // initializare
    glutInit(&argc, argv);

    // setarea modului pentru afisare
    glutInitDisplayMode(mode: GLUT_DOUBLE | GLUT_RGBA | GLUT_DEPTH);

    // specificare dimensiune fereastra
    glutInitWindowSize(windowWidth, windowHeight);

    // creare fereastra cu numele specificat
    glutCreateWindow("Bile Saltarete - DraghiciCiontu - CR4S1A");

    // apelare functii
    glutDisplayFunc(func: display);
    glutReshapeFunc(func: reshape);
    glutKeyboardFunc(func: keyboard);
    glutSpecialFunc(func: special_keyboard);

    // se spcifica perioada de timp dintre fiecare apel al functiei
    glutTimerFunc(millis: 0, func: timer, value: 0);
    glEnable(cap: GL_DEPTH_TEST);

    // incarcare textura bila
    loadTexture();

    // rulare program
    glutMainLoop();

    return 0;
}
```

Output:



Ciontu Claudia-Elena
Draghici Andreea-Maria
CR4.S1A

PREZENTAREA FINALA

Pentru ultima prezentare am modificat codul astfel incat sa creez un plan 3D pe care bilele sa sara, am adaugat inca o bila, am schimbat unghiul camerei incat obiectele sa pot fi privite in spatial 3D si am modificat culorile obiectelor, planului si ferestrei.

Am pastrat vairbilele care reprezinta unghiurile camerei in sistemul de coordinate 3D pentru a pozitiona si roti camera in spatiu. La fel si in cazul variabilelor ce reprezinta pozitia bilelor in spatiu. Avem o variabila ballMove de tipul Boolean care indica deirectia de miscare a bilei.

```
#include <windows.h>
#include <GL/glut.h>
#include <iostream>
#include <math.h>

/* reprezinta unghiurile camerei intr-un sistem de coordonate 3D */
double cameraAngleX = 45;
double cameraAngleY = 2.5;
double cameraAngleZ = 40;

/* reprezinta pozitia unei bile în spatiu */
double ballPosX = 3;
double ballPosY = 3;
double ballPosZ = 3;

/* variabile utilizate pentru stocarea dimensiunii ferestrei grafice */
int windowWidth = 800;
int windowHeight = 600;
double width = 5.0;
double height = 8.5;

/* este o variabila de tip bool care indica directia de miscare a formei geometrice */
bool ballMove = true;
```

Functia de initializare pentru afisarea grafica sub format 3D.

```
/* initializarea contextului OpenGL */
void initGL() {
    glClearColor(red: 0, green: 1, blue: 1, alpha: 0); //seteaza culoarea de fundal
    glShadeModel(mode: GL_FLAT); //seteaza modul de shading al bilelor
    glEnable(cap: GL_DEPTH_TEST); //activare test de adancime
}
```

Functia de afisare a bilelor, planului in fereastra. Initial stergem buffer-ul de culoare pentru a curate fereastra înainte de a desena bilele in scena. Apoi setam modul de transformare a matricei, adica orice operatie ulterioara se va aplica asupra modelului si vizualizarii in acelasi timp.

Resetam matricea curenta la matricea identitate, adica matricea in care niciun obiec nu este transformat. Apoi cu funcție gluLookAt() setam pozitia camerei in spatial 3D si directia de vizualizarea.

Ciontu Claudia-Elena
Draghici Andreea-Maria
CR4.S1A

Pentru a desena astfel reteaua de coordonate si bile folosim in prim-plan doua bucle for pentru a desena o serie de linii paralele pentru fiecare axa. In cele din urma, bilele sunt denesate folosind funcție glutSolidSphere() si fransformate cu glTranslatef() pentru a le plasa in pozitia dorita in spatial 3D.

```
/* functie de afisare a obiectelor in fereastra */
void display(void) {

    // se curata buffer-ul de culoare
    glClear(mask: GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    // seteaza matricea de modelare si vizualizare
    glMatrixMode(GL_MODELVIEW);//seteaza modul de transformare al matericei
    glLoadIdentity();//resetare matrice curenta la matricea identitate

    // folosita pentru a defini pozitia camerei cu ajutorul parametrilor de distanta , unghi de vedere si pozitia obiectului
    gluLookAt(eyex: -10 + cameraAngleX, eyey: 9 + cameraAngleY, eyez: -10 + cameraAngleZ,
        centerx: ballPosX, centery: ballPosY, centerz: ballPosZ,
        upx: 0, upy: 6, upz: 0);


    /* fiecare bucla for deseaza o serie de linii paralele pentru fiecare axa */
    for (double i = -20; i < 20; i += 1.1) {
        glBegin(mode: GL_QUADS);
        glColor3f(red: 1.5f, green: 1.0f, blue: 1.0f); // plan galben
        glVertex3f(x: -20, y: 0, z: -20);
        glVertex3f(x: -20, y: 0, z: 20);
        glVertex3f(x: 20, y: 0, z: 20);
        glVertex3f(x: 20, y: 0, z: -20);
        glEnd();

    }

    for (double i = -20; i < 20; i += 1.1) {
        glBegin(mode: GL_QUADS);
        glColor3f(red: 1.5f, green: 1.0f, blue: 1.0f); // plan galben
        glVertex3f(x: -20, y: 0, z: -20);
        glVertex3f(x: -20, y: 0, z: 20);
        glVertex3f(x: 20, y: 0, z: 20);
        glVertex3f(x: 20, y: 0, z: -20);
        glEnd();
    }

    // desenare bile
    glTranslatef(x: 7, y: width + 1.5, z: 4);
    glColor3f(red: 1, green: 0, blue: 0);//red
```

Cu ajutorul functiei glPopMatric() inlaturam matricea curenta din stica matricelor, facand ca matricea anterioare sa devina matricea curenta. Iar cu glutSwapBuffers() schimbam bufferul de fata cu cel din spate, ceea ce face ca obiectele sa fie afisate pe ecran.

```
    // desenare bile
    glTranslatef(x: 7, y: width + 1.5, z: 4);
    glColor3f(red: 1, green: 0, blue: 0);//red
    glutSolidSphere(radius: 1.2f, slices: 50, stacks: 50);
    glEnd();

    glTranslatef(x: 7, y: width + 1, z: 9);
    glColor3f(red: 0, green: 1, blue: 0); // green
    glutSolidSphere(radius: 1.2f, slices: 50, stacks: 50);
    glEnd();

    glPopMatrix();

    // schimbare buffer pentru a permite afisarea
    glutSwapBuffers();
}
```

Ciontu Claudia-Elena
Draghici Andreea-Maria
CR4.S1A

```
/* apelata cand fereastra de afisare este redimensionata
   scop pentru a adapta proiectia proportiilor obiectului afisat pentru a nu se deforma in cazul schimbarii dimensiunii ferestrei */
void reshape(int w, int h) {

    // specificam noua zona de afisare a ferestrei, aceasta fiind impartita in regiuni de coordonate
    glViewport(x: 0, y: 0, width: w, height: h);

    // se seteaza matricea de proiectie
    glMatrixMode(mode: GL_PROJECTION);
    glLoadIdentity();

    // se specifica campul vizual al camerei in grade, raportul de aspect al ferestrei,
    // precum si distanta dintre planul apropiat si cel indepartat
    gluPerspective(fovy: 45.0, aspect: (float)w / (float)h, zNear: 0.1, zFar: 100.0);
}
```

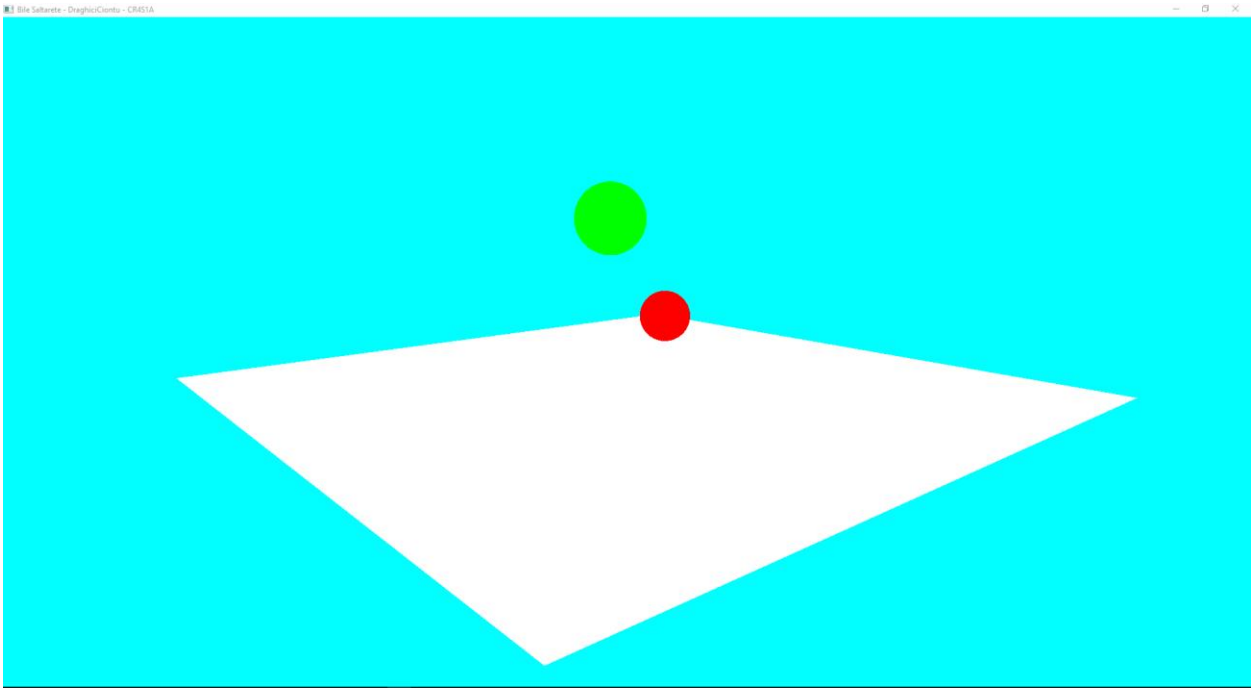
```
void keyboard(unsigned char key, int x, int y) {
    switch (key) {
    case 27:
        exit(_Code: 0); // apasare tasta ESC iar programul se inchide
        break;
    case 'd':
        cameraAngleX += 0.2;
        break;
    case 'a':
        cameraAngleX -= 0.2;
        break;
    case 'l':
        ballPosX += 0.8;
        break;
    case 'j':
        ballPosX -= 0.2;
        break;
    case 'q':
        cameraAngleY += 0.2;
        break;
    case 'e':
        cameraAngleY -= 0.2;
        break;
    case 'u':
        ballPosY += 0.2;
        break;
    case 'o':
        ballPosY -= 0.2;
        break;
    case 'w':
        cameraAngleZ += 0.2;
        break;
    case 's':
        cameraAngleZ -= 0.2;
        break;
    case 'i':
        ballPosZ += 0.8;
        break;
    case 'k':
        ballPosZ -= 0.8;
        break;
    }
    glutPostRedisplay();
}
```

Ciontu Claudia-Elena
Draghici Andreea-Maria
CR4.S1A

```
/* pentru tastele speciale */
void special_keyboard(int key, int x, int y) {

    // pentru a determina care tasta a fost apasata
    switch (key) {
    case GLUT_KEY_UP:
        cameraAngleX -= 0.2f; // se decrementeaza unghiul de rotatie al camerei in jurul axei X
        break;
    case GLUT_KEY_DOWN:
        cameraAngleX += 0.2f; // se incrementeaza unghiul de rotatie al camerei in jurul axei X
        break;
    case GLUT_KEY_LEFT:
        cameraAngleY -= 0.2f; // se decrementeaza unghiul de rotatie al camerei in jurul axei Y
        break;
    case GLUT_KEY_RIGHT:
        cameraAngleY += 0.2f; // se incrementeaza unghiul de rotatie al camerei in jurul axei Y
        break;
    default:
        break;
    }
}

void timer(int v) {
    // determina redesenarea ecranului in urmatorul cadru de afisare
    glutPostRedisplay();
    glutTimerFunc(millis: 16, func: timer, value: 0);
}
```

Animam bila care sare in sus si in jos pe ecran. Verificam daca latimea este 0 sau daca bila atinge marginea de sus sau de jos a ecranului. Astfel se inverseaza Valoarea variabilei, ceea ce inseamna ca bila isi va schimba directia in urmatoarea actualizare. Al doilea if actualizeaza pozitia mingii. In cele din urma, funcție utilizează glutTimerFunc() pentru a apela funcție timer cu un interval de 30 de milisecunde. Acest lucru asigura ca funcție bouncing_balls() va fi apelata la fiecare 30 ms, ceea ce face ca bila sa se miste intr-un mod continuu si animat pe ecran.

```
void bouncing_balls() {

    /* verificam daca mingea atinge marginea de sus sau de jos a ecranului */
    if (width == 0 || width == height) {
        //se inverseaza valoarea variabilei, iar mingea isi schimba directia
        ballMove ^= 1;
    }

    /* actualizeaza pozitia bilei */
    if (ballMove) {
        width += 0.5; // bila se ridica
    }
    else {
        width -= 0.5; // bila coboara
    }

    glutTimerFunc(millis: 30, func: timer, value: 0);
}
```

Ciontu Claudia-Elena
Draghici Andreea-Maria
CR4.S1A

Output: